Linux Commands

1. Terminal and Shell Fundamentals

These commands help you work with the terminal, shell, and basic system information.

Command / Operator	Description	Example
pwd	Displays the current working directory.	pwd → /home/student
whoami	Displays the currently logged-in username.	whoami → student
hostname	Displays the computer/system name.	hostname
clear	Clears the terminal screen.	clear
history	Displays previously executed commands.	history
echo	Displays text or a variable value.	echo "Hello Linux"
man	Opens the manual/help page for a command.	man ls
date	Displays the current date and time.	date
uname	Displays basic OS/kernel information.	uname
uname -a	Displays detailed system information.	uname -a

2. File System Navigation

These commands help you move around the Linux file system and locate files and directories.

Command / Operator	Description	Example
pwd	Displays the current directory path.	pwd
ls	Lists files and directories.	ls
ls -l	Lists files with detailed information such as permissions, owner, size and time.	ls -l
ls -a	Lists all files, including hidden files.	ls -a
cd	Changes the current directory.	cd Documents
cd ..	Moves to the parent directory.	cd ..
cd ~	Moves to the user's home directory.	cd ~
cd /	Moves to the root directory.	cd /
tree	Displays files/directories in a tree structure.	tree
find	Searches for files and directories.	find . -name "notes.txt"

3. File and Directory Operations

These commands create, copy, move, rename and delete files and directories.

Command / Operator	Description	Example
touch	Creates a new empty file.	touch notes.txt
mkdir	Creates a directory.	mkdir Documents
mkdir -p	Creates a directory and required parent directories.	mkdir -p project/src/java
cp	Copies a file.	cp notes.txt backup.txt
cp -r	Copies a directory and its contents	cp -r Documents DocumentsBackup

	recursively.	
mv	Moves or renames a file/directory.	mv old.txt new.txt
rm	Deletes a file.	rm notes.txt
rm -r	Deletes a directory and its contents.	rm -r Documents
rmdir	Deletes an empty directory.	rmdir Test

4. File Viewing and Editing

These commands help you read, inspect and edit files.

Command / Operator	Description	Example
cat	Displays the complete contents of a file.	cat notes.txt
less	Views a file one screen at a time; press q to exit.	less largefile.txt
more	Views file contents page by page.	more largefile.txt
head	Displays the beginning of a file; normally first 10 lines.	head notes.txt
head -n 5	Displays the first 5 lines.	head -n 5 notes.txt
tail	Displays the end of a file; normally last 10 lines.	tail notes.txt
tail -n 5	Displays the last 5 lines.	tail -n 5 notes.txt
tail -f	Continuously displays new content added to a file; useful for logs.	tail -f application.log
nano	Opens a simple terminal text editor.	nano notes.txt
vim	Opens the Vim text editor.	vim notes.txt
vi	Opens the Vi text editor.	vi notes.txt
wc	Counts lines, words and bytes.	wc notes.txt
wc -l	Counts lines.	wc -l notes.txt
wc -w	Counts words.	wc -w notes.txt
wc -c	Counts bytes.	wc -c notes.txt

5. File Permissions and Security

Linux controls access to files and directories using permissions for the owner, group and others.

Command / Operator	Description	Example
ls -l	Displays permissions and ownership.	ls -l
chmod	Changes file/directory permissions.	chmod 644 notes.txt
chown	Changes the owner.	sudo chown john notes.txt
chgrp	Changes group ownership.	sudo chgrp developers notes.txt
umask	Displays or sets default permissions for new files/directories.	umask / umask 027
sudo	Executes a command with elevated privileges.	sudo apt update
su	Switches to another user.	su john
id	Displays user ID and group information.	id

Permission Symbols

Command / Operator	Description	Example
r	Read permission.	r--
w	Write permission.	w--
x	Execute permission.	x--
-	Permission is not granted.	---

Example: -rwxr-xr-- means user = rwx, group = r-x, others = r--.

6. Process and Task Management

A process is a running program. These commands help you view and manage processes and shell jobs.

Command / Operator	Description	Example
ps	Displays processes associated with the current shell/session.	ps
ps aux	Displays detailed information about running processes.	ps aux
top	Shows processes and CPU/memory usage in real time; press q to exit.	top
htop	Interactive process viewer; may need installation.	htop
kill	Sends a signal to a process using its PID.	kill 1234
kill -9	Forcefully terminates a process; use carefully.	kill -9 1234
pkill	Terminates processes by name.	pkill firefox
jobs	Shows jobs running in the current shell.	jobs
bg	Continues a stopped job in the background.	bg
fg	Brings a background job to the foreground.	fg
nohup	Runs a command so it can continue after logout/session closure.	nohup ./server.sh &
sleep	Pauses execution for a specified time.	sleep 5

7. Input/Output Redirection and Pipes

Redirection changes where a command gets input or sends output. A pipe connects one command's output to another command's input.

Command / Operator	Description	Example
>	Redirects standard output to a file and overwrites it.	ls > files.txt
>>	Redirects output and appends to a file.	echo "Hello" >> notes.txt
<	Takes standard input from a file.	wc -l < notes.txt
2>	Redirects standard error to a file.	ls abc 2> error.txt
&>	Redirects both standard output and errors in Bash.	command &> output.txt
|	Sends one command's output as another command's input.	ls | grep ".txt"
tee	Displays output and simultaneously writes it to a file.	ls | tee files.txt
xargs	Uses input to build and execute another command.	echo "a.txt b.txt" | xargs rm

Pipe Example

ls | grep ".txt" — ls produces the file list, the pipe passes it to grep, and grep filters entries containing .txt.

8. Environment Variables and Configuration

Environment variables store configuration information used by the shell and programs.

Command / Operator	Description	Example
env	Displays environment variables.	env
printenv	Displays environment variables or one selected variable.	printenv HOME

echo $PATH	Displays PATH, the directories searched for executables.	echo $PATH
export	Creates or modifies an environment variable.	export COURSE="Linux"
unset	Removes an environment variable.	unset COURSE
set	Displays shell variables, functions and related information.	set
source	Reads and executes commands from a file in the current shell.	source ~/.bashrc
alias	Creates a shortcut for a command.	alias ll='ls -l'
unalias	Removes an alias.	unalias ll

Common Configuration Files

File	Description	Example
~/.bashrc	Bash configuration for the current user; commonly used for aliases and shell settings.	nano ~/.bashrc
~/.profile	User login configuration file.	nano ~/.profile
/etc/environment	System-wide environment configuration file.	sudo nano /etc/environment

9. Package Management

Package managers install, update, search and remove software. Ubuntu/Debian commonly use apt; Fedora/RHEL commonly use dnf.

Command / Operator	Description	Example
apt update	Updates local package information from configured repositories.	sudo apt update
apt upgrade	Upgrades installed packages.	sudo apt upgrade
apt install	Installs a package.	sudo apt install git
apt remove	Removes an installed package.	sudo apt remove tree
apt search	Searches available packages.	apt search python
apt show	Displays package information.	apt show git
apt autoremove	Removes packages no longer required.	sudo apt autoremove
dnf install	Installs a package on Fedora/RHEL.	sudo dnf install git
dnf update	Updates installed packages.	sudo dnf update
dnf remove	Removes a package.	sudo dnf remove git
dnf search	Searches for packages.	dnf search java

10. Build and Execute Workflow

These commands are commonly used to compile, build and execute programs.

Command / Operator	Description	Example
gcc	Compiles C source programs.	gcc hello.c -o hello
g++	Compiles C++ source programs.	g++ hello.cpp -o hello
make	Automates a build process using instructions in a Makefile.	make
javac	Compiles Java source code into .class bytecode.	javac Hello.java
java	Runs a compiled Java program.	java Hello
./program	Executes a program located in the current directory.	./hello
which	Shows the path of an executable found through PATH.	which java
chmod +x	Adds execute permission to a file.	chmod +x script.sh

Java Build and Execute Workflow

Hello.java → javac → Hello.class → java → Program Output

11. Complete Practical Exercise

Students can perform the following sequence to practice most of the commands covered in this guide.

Step 1 — Find where you are

```
pwd

whoami

hostname

date
```

Step 2 — Explore the file system

```
ls

ls -l

ls -a

cd ~

pwd
```

Step 3 — Create files and directories

```
mkdir linux-practice

cd linux-practice

touch file1.txt file2.txt

mkdir documents

ls
```

Step 4 — Add and view file content

```
echo "Linux is an operating system" > file1.txt

cat file1.txt

echo "Linux commands are powerful" >> file1.txt

cat file1.txt
```

Step 5 — Copy and rename

```
cp file1.txt backup.txt

mv file2.txt file2-renamed.txt

ls
```

Step 6 — Practice pipes

```
ls | grep ".txt"
```

Step 7 — Practice redirection

```
ls > files.txt

cat files.txt
```

Step 8 — Check and change permissions

```
ls -l

chmod +x file1.txt

ls -l
```

Step 9 — Practice process management

```
sleep 100

Ctrl + Z

bg

jobs

fg
```

Step 10 — Practice environment variables

```
echo $PATH

export COURSE="Linux"

echo $COURSE

unset COURSE

echo $COURSE
```

12. Important Notes for Beginners

- Linux commands are generally case-sensitive. ls and LS are different.
- Use man command to learn about a command, for example: man chmod.
- Be careful with rm and rm -r because deleted files are not normally moved to a recycle bin.
- Use sudo only when administrative privileges are actually required.
- The current directory is represented by . and the parent directory by ..
- The root directory is /, while the current user's home directory is represented by ~.
- Use Ctrl + C to stop many foreground commands and q to exit programs such as less and top.